BIG DATA WITH PYTHON · SESSION 2

Analyzing Data with Pandas

Select, compute, filter, sort and group a real dataset, one small step at a time.

Notebook: BigData-session-2.ipynb · Data: sherlock/sherlock_mystery.csv

QUICK RECAP

A DataFrame is a table

Rows are records: one snapshot of an app running on a phone.

Columns are features: CPU use, memory, threads…

Each column on its own is a **Series**.

Our **df_mystery** table has 200 rows and 14 columns, from two apps: Facebook and WhatsApp.

ApplicationName	CPU_USAGE	num_threads
WhatsApp	10.0	100
Facebook	0.9	72
Facebook	0.0	68

A few rows and columns, like the example in the notebook

TODAY'S TOOLKIT

Six building blocks

1 · Select

Keep some columns

`df[['a', 'b']]`

2 · Drop

Remove columns or rows

`df.drop(...)`

3 · Compute

Make new columns

`df['a'] * 2`

4 · Filter

Keep rows that match

`df[df['a'] > 5]`

5 · Sort

Put rows in order

`df.sort_values(...)`

6 · Group

Summarize per category

`df.groupby(...)`

1 · SELECT COLUMNS

Keep only the columns you need

```
# one column → a Series
df_mystery['vsize']

# a list of columns → a DataFrame
df_memstats = df_mystery[['ApplicationName', 'vsize', 'Mem']]
```

Two pairs of brackets. The outer [ ] selects; the inner [ ] is a normal Python list of names.

Names must match exactly, including capital letters. Check them with df.columns.

2 · DROP

Remove columns or rows you don’t need

```
df2 = df_mystery.drop('Unnamed: 0', axis=1) # drop a column
df3 = df_mystery.drop([0, 1, 2], axis=0) # drop rows 0, 1, 2
df_mystery.drop('Unnamed: 0', axis=1, inplace=True) # change df_mystery itself
```

axis=0 → rows

Give row labels

axis=1 → columns

Give column names

inplace=True

Changes the original. Without it you get a new table.

3 · COMPUTE

Do math on a whole column at once

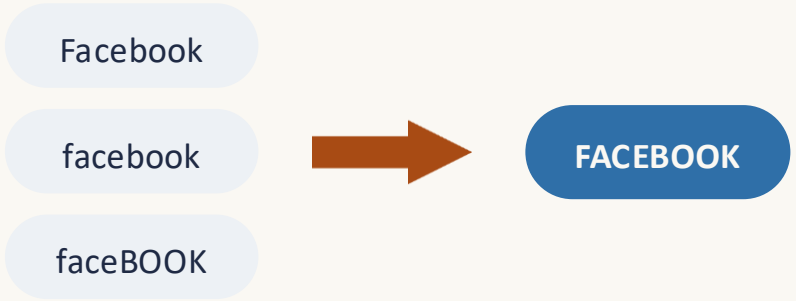
```
# bytes → gigabytes (1e-9 means 0.000000001)
df_mystery['vsize_gb'] = df_mystery['vsize'] * 1e-9

# combine two columns, row by row
df_mystery['utime_self'] = df_mystery['utime'] - df_mystery['guest_time']
```

No loop needed. Pandas does the math for every row.

A new name on the left creates a new column. An existing name replaces it.

Clean up messy text with .str



To a computer these are three different words. Make them match before you count or group.

```
df_mystery['ApplicationName'] = df_mystery['ApplicationName'].str.upper()
# also: .str.lower()
```

3 · COMPUTE: YOUR OWN RULE

Write a function, then apply() it

```
# time can’t be negative: turn negatives into 0
def non_negative(val):
    if val > 0:
        return val
    else:
        return 0

fixed = df['guest_time'].apply(non_negative)
```

apply() calls your function **once for every value** in the column.

Pass the function’s **name**, without ( ).

Built-in pandas tools are faster. Use apply() when none fits.

4 · FILTER

Keep only the rows that answer “yes”

Step 1 · Ask

```
df['CPU_USAGE'] > 2
```

Gives True or False for every row.

Step 2 · Keep

```
df[df['CPU_USAGE'] > 2]
```

Put the question inside [ ]. Only True rows stay.

Comparisons: `== equal` `!= not equal` `> >=` `< <=`

4 · FILTER: TWO CONDITIONS

Combine questions with & and |

```
# Facebook rows AND CPU above 0.5
df[(df['ApplicationName'] == 'Facebook') & (df['CPU_USAGE'] > 0.5)]

# Facebook rows OR CPU above 0.5
df[(df['ApplicationName'] == 'Facebook') | (df['CPU_USAGE'] > 0.5)]
```

& means AND

Both must be true

| means OR

At least one is true

Watch out

Wrap each question in ( ). Don’t use the words and / or.

5 · SORT

Sort to find the biggest users

```
# highest CPU first, show the top 20
df.sort_values(by='CPU_USAGE', ascending=False).head(20)

# sort by threads; break ties with CPU
df.sort_values(by=['num_threads', 'CPU_USAGE'], ascending=False)
```

ascending=False puts the biggest values first.

A list in by= sorts by the first column, then the next.

Filter first, then sort to answer sharper questions.

AGGREGATE

Summarize many values in one number

Method	Gives you
.max() / .min()	largest / smallest
.mean()	average
.median()	middle value
.sum()	total
.mode()	most common value
.count()	how many values

```
df_mystery['cutime'].median()
df_mystery['ApplicationName'].mode()
df_mystery.mean(numeric_only=True)
```

On one column you get one number. On the whole table you get one number **per column**.

6 · GROUP

Group by: split, apply, combine




```
df.groupby('ApplicationName')['vsize'].max()
df.groupby('ApplicationName').size() # rows per app
```

Two ways to put tables together

concat: stack rows



More rows of the same kind go underneath.

```
pandas.concat([df1, df2])
```

merge: match side by side



Extra columns join on a shared key, like the app name.

```
df.merge(df_appinfo, how='left',
      on='ApplicationName')
```

DEBUGGING HELP

Common beginner mistakes

INSTEAD OF	WRITE THIS
df['a', 'b']	df[['a', 'b']]
df[df['x'] > 1 & df['y'] < 5]	df[(df['x'] > 1) & (df['y'] < 5)]
(cond1) and (cond2)	(cond1) & (cond2)
df.drop('a', axis=1)	df = df.drop('a', axis=1)
df['cpu_usage'] → KeyError	df['CPU_USAGE'] (check df.columns)

KEY POINTS

Your pandas toolkit

SELECT

df[['a', 'b']]

DROP

df.drop('a', axis=1)

COMPUTE

df['a'] * 2 · .str.upper() · .apply(f)

SUMMARIZE

.mean() .median() .max() .sum()

FILTER

df[(cond1) & (cond2)]

SORT

df.sort_values(by='a', ascending=False)

GROUP

df.groupby('app')['a'].mean()

COMBINE

pandas.concat([...]) · df.merge(...)

Next session: cleaning and visualizing a much bigger dataset